



NAME OF THE PROJECT :- LIBRARY MANAGEMENT SYSTEM

NAME :- PARAS VATS

REGISTRATION NUMBER :- 26BCE11515

COURSE :- INTRODUCTION TO PROBLEM SOLVING

COURSE CODE :- CSE1021

Table of Contents

1. Executive Summary & Design Scope	Page 2
2. Architectural Foundations & Decompositions	Page 3
3. Memory Model & Relational Simulation Schema	Page 4
4. Core Functional Modules & Pipeline Validation	Page 5
5. Systematic Source Code Ledger - Core Base	Page 6
6. Systematic Source Code Ledger - Application Layer	Page 7
7. Algorithmic Integrity, Threat Mitigation & Roadmap	Page 8

1. Executive Summary

The contemporary development of utility enterprise applications frequently defaults to pulling massive third-party library stacks and heavyweight persistence layer engines. This report comprehensively analyses a stark counter-alternative: an encapsulated, high-efficiency, zero-dependency **Library Management System** built entirely inside the native Python 3.x structural runtime ecosystem. Designed purposefully as an explicit educational benchmark, this framework implements full transaction mechanics, safe validation loops, relational linkage simulations, and error-trapped execution workflows without straying from standard native objects.

By utilizing optimized standard list collections combined with string-indexed compound dictionary registries, the architecture simulates an transactional database workspace in volatile memory. All operational mutations preserve state stability through targeted criteria validation routines. This documentation captures the comprehensive lifecycle profile, memory blueprints, architectural patterns, and structured implementations governing this ultra-lightweight system framework.

2. Architectural Foundations & Decompositions

The system strictly leverages a decoupled functional pipeline mapping layout to guarantee separation of programmatic concerns while routing workflow state mutations through isolated validation nodes. Despite being contained within a singular executable runtime module, the system explicitly abstracts its visual terminal representation away from core state manipulation vectors.

Functional Decomposition Architecture

The execution lifecycle operates through three distinct layout layers: User Interface Routing, Logical Validation Pipeline, and Volatile State Registry. The console choice engine intercepts user choices and maps them to respective execution frames, passing parameters downstream where validation safeguards ensure strict domain formatting rules before affecting runtime state records.

Structural Layer	Functional Components Included	Operational Execution Responsibility
User Interface Routing	print_menu() main() loop	Manages command line interaction loop, parses user inputs, and maps execution contexts via function reference routers.
Validation Pipeline	add_book() validation issue_book() transaction locks	Enforces strict business rules: type verification, null checking, inventory boundaries, and uniqueness guards.
Volatile State Layer	Global books catalog registry find_book_by_id() locator	Provides low-latency reference, index scans, row extraction vectors, and pointer modification within memory allocations.

3. Memory Model & Relational Simulation Schema

Because the database operates inside dynamic RAM allocations, the data modeling strategy relies on deep nested structures utilizing compound key definitions. This structure allows the program to simulate complex real-world data relationships without relational structural backings.

Compound Schema Structure Ledger

Key Name	Primitive Data Type	Relational Mapping & Integrity Objective
id	int (Scalar)	Primary Key indicator. Uses a strict auto-increment loop controlled by a global state sequence register to provide single-record index lookups.
title	str (Normalized)	Stores the alphanumeric textual representation of the item name. Enforces implicit non-null validation layers.
author	str (Normalized)	Captures primary creator data. Used together with title tokens during comprehensive keyword linear evaluation sweeps.
copies	int (Positive)	Defines total volume depth limits for an entry. Serves as the absolute upper boundary ceiling for ongoing transactions.
issued_to	list [str]	Simulates a direct relational link field by capturing and storing variable borrower unique strings. Tracks circulation volume via active list metrics.

Dynamic Inventory Allocation Matrix

Inventory states remain fully calculated at execution runtime rather than statically duplicated across database elements. This completely avoids dangerous sync issues. The standard mathematical algorithm determining item volume availability translates as follows:

Available Inventory Copies = book['copies'] - len(book['issued_to'])

4. Core Functional Modules & Pipeline Validation

The operational dependability of an interactive catalog program depends entirely on blocking bad inputs before they can introduce system state errors. Every individual module uses specialized validation patterns to maintain integrity.

4.1 Content Registration Subsystem

The book addition process screens out blank text fields through whitespace trimming algorithms (.strip()). Numerical parameters undergo string digit matching operations via .isdigit() to block negative figures and float numbers, protecting integer parameters across runtime loops.

4.2 Circulation Processing & Relational Transaction Locks

Step	Validation Boundary Process	System Failure Action / Error Handling
01	Lookup Book ID via find_book_by_id	Halts process if return reference matches None.
02	Evaluate available pool (Copies - Issued)	Rejects transaction if active pool depth equals zero.
03	Scan borrower string array for duplicates	Blocks operation if user already holds an active copy.

5. Systematic Source Code Ledger - Core Base

```
# Volatile Memory Space Data Structure Allocation
books = []
next_id = 1

def add_book():
    global next_id
    title = input("Enter book title: ").strip()
    author = input("Enter author name: ").strip()
    if title == "" or author == "":
        print("■ Title and author cannot be empty.\n")
        return
    copies_input = input("Enter number of copies: ").strip()
    if not copies_input.isdigit() or int(copies_input) <= 0:
        print("■ Number of copies must be a positive whole number.\n")
        return
    book = {
        "id": next_id,
        "title": title,
        "author": author,
        "copies": int(copies_input),
        "issued_to": []
    }
    books.append(book)
    print(f"■ Book '{title}' added successfully with ID {next_id}.\n")
    next_id += 1

def view_books():
    if not books:
        print("■ No books available in the library.\n")
        return
    print("\n" + "-" * 60)
    print(f"{'ID':<5}{'Title':<20}{'Author':<15}{'Available':<10}")
    print("-" * 60)
    for book in books:
        available = book["copies"] - len(book["issued_to"])
        print(f"{book['id']:<5}{book['title']:<20}{book['author']:<15}{available:<10}")
    print("-" * 60 + "\n")
```

6. Systematic Source Code Ledger - Application Layer

```
def search_book():
    keyword = input("Enter title or author to search: ").strip().lower()
    if keyword == "":
        print("■ Search keyword cannot be empty.\n")
        return
    results = [b for b in books if keyword in b["title"].lower() or keyword in b["author"].lower()]
    if not results:
        print("■ No matching books found.\n")
    else:
        print(f"\n■ Found {len(results)} matching book(s):")
        for book in results:
            available = book["copies"] - len(book["issued_to"])
            print(f"   ID {book['id']}: '{book['title']}' (Available: {available}/{book['copies']})"
        )

def issue_book():
    try:
        book_id = int(input("Enter Book ID to issue: ").strip())
    except ValueError:
        return
    book = find_book_by_id(book_id)
    if book and (book["copies"] - len(book["issued_to"]) > 0):
        borrower = input("Enter name: ").strip()
        if borrower and borrower not in book["issued_to"]:
            book["issued_to"].append(borrower)
```

7. Algorithmic Integrity, Threat Mitigation & Roadmap

7.1 Runtime Complexity Analysis

Because data is organized using standard dynamic list structures, index-based access commands like `find_book_by_id` run on linear sequential search times, scaling at an $O(N)$ notation where N represents total record depth. While optimal for small to medium setups, massive dataset growth can be optimized by migrating target items into indexed hash-map configurations to achieve $O(1)$ performance lookup metrics.

7.2 Scalability Roadmap Evolution

Phase	Target Architecture Upgrade Focus	Implementation Vector
Phase 1	Flat-File Persistence Layer Storage	Integrate native json or csv modules to write states to disk during shutdown sequences.
Phase 2	Relational Data Base Conversion	Migrate schema structures to an embedded sqlite3 engine using raw SQL queries.
Phase 3	Concurrent Client-Server Framework	Expose endpoints via a lightweight FastAPI REST API network wrapper.
Phase 4	Modern Graphical User Interface	Replace text inputs with a responsive Tkinter frontend interface.

This is for informational purposes only. AI responses may include mistakes.